

Автономная некоммерческая профессиональная образовательная
организация
«УРАЛЬСКИЙ ПРОМЫШЛЕННО-ЭКОНОМИЧЕСКИЙ ТЕХНИКУМ»

Отчет о проектной работе

Дисциплина: МДК 02.01. Технология
разработки программного обеспечения

К 09.02.07 406 6 26

Выполнила команда 6

Студенты:

Мусин Дамир Рашидович

Бараковских Александр Анатольевич

Дмитриев Роман Николаевич

Семеновская Елена Александровна

Куратор

_____ Копотев Н.В.

« __ » _____ 2026 г.

Екатеринбург

2026

Содержание

1. ВВЕДЕНИЕ.....	4
1.1. Описание проекта	4
1.2. Актуальность	4
1.3. Цель проекта	4
1.4. Задачи	5
2. КОМАНДА	6
3. ЦЕЛЕВАЯ АУДИТОРИЯ	7
3.1. Основные группы пользователей.....	7
3.2. Потребности аудитории	7
3.3. Сценарии использования	7
4. Подбор средств для разработки	8
4.1. Язык программирования и платформа	8
4.2. Среда разработки и инструменты сборки	8
4.3. Архитектурный паттерн и проектирование интерфейса	8
4.4. Работа с данными и сетевым взаимодействием	9
4.5. Вспомогательные инструменты и документирование.....	9
5. АНАЛИЗ АНАЛОГОВ	10
5.1. Существующие решения.....	10
5.2. Конкурентные преимущества ZenQuotesApp	11
5.3. Уникальные особенности	11
6. ПРОЦЕСС РАЗРАБОТКИ	12
6.1. Архитектура приложения	12
6.2. Основные компоненты	12
6.3. Этапы разработки	13
6.4. Обработка ошибок	13
6.5. Оптимизации.....	14
7. MVP (MINIMUM VIABLE PRODUCT).....	15
7.1. Определение MVP	15
7.2. Функции MVP	15
7.3. Функции пост-MVP (реализовано).....	15
7.4. Критерии успеха MVP.....	15
7.5. Критерии эффективности и метрики MVP.....	15
8. ПРИМЕР ИСПОЛЬЗОВАНИЯ	16
8.1. Установка и запуск.....	16
8.2. Сценарий 1: Первый запуск.....	16
8.3. Сценарий 2: Просмотр случайной цитаты	16

8.4. Сценарий 3: Поиск по ключевому слову.....	17
8.5. Сценарий 4: Игра "Угадай автора"	18
8.6. Сценарий 5: Принудительное обновление	18
8.7. Сценарий 6: Работа без интернета	19
8.8. Структура файла хранилища	19
9. ЗАКЛЮЧЕНИЕ.....	20
9.1. Итоги разработки	20
9.2. Технические особенности проекта	20
9.3. Практическая ценность	20
9.4. План дальнейшего развития проекта	20

ZenQuotes App - Генератор мотивирующих цитат

1. ВВЕДЕНИЕ

1.1. Описание проекта

ZenQuotesApp – десктопное WPF-приложение на C# (.NET 8), реализованное по паттерну MVVM. Приложение взаимодействует с внешним REST-API zenquotes.io, асинхронно загружает коллекции вдохновляющих цитат, сохраняет их в локальной базе данных SQLite и предоставляет пользователю графический интерфейс в тёмной теме, стилизованной под Spotify. Поддерживает автономную работу, поиск, интерактивную викторину и гибкую настройку запросов.

1.2. Актуальность

В современном цифровом мире качественные цитаты широко используются в образовательных материалах, презентациях, социальных сетях и личной мотивации. Приложение решает проблему быстрого, удобного и стабильного доступа к верифицированному контенту, гарантируя работоспособность даже при отсутствии интернета или ограничении API.

1.3. Цель проекта

Создание отзывчивого десктопного приложения с современной архитектурой (MVVM + SQLite), которое:

Предоставляет случайные и тематические цитаты через асинхронный интерфейс.

Обеспечивает офлайн-доступ за счёт локального кэширования.

Реализует интерактивный игровой режим для обучения и развлечения.

Гарантирует отказоустойчивость (API → БД → резервный пул) и возможность отмены долгих операций.

1.4. Задачи

1. Интеграция с внешним API ZenQuotes с обработкой лимитов и таймаутов.
2. Проектирование и реализация слоя данных на SQLite с дедупликацией и транзакциями.
3. Построение MVVM-архитектуры: ViewModelBase, RelayCommand, привязки данных, навигация по разделам.
4. Разработка асинхронного оркестратора загрузки с прогрессом (IProgress) и отменой (CancellationToken).
5. Создание WPF-интерфейса: кастомное окно (WindowChrome), виртуализация списков, единая тема и стилизация.
6. Реализация модуля настроек с сериализацией в JSON.

2. КОМАНДА

Команда №6

Состав команды:

Бараковский Александр Анатольевич

Дмитриев Роман Николаевич

Мусин Дамир Рашидович

Семеновская Елена Александровна

Распределение ролей:

Анализ требований и проектирование архитектуры, :

- Бараковский Александр Анатольевич

Реализация WPF UI, MVVM и стилизация:

- Бараковский Александр Анатольевич

Разработка backend-логики и сервисов:

- Мусин Дамир Рашидович
- Бараковский Александр Анатольевич

Интеграция с API и работа с сетью:

- Мусин Дамир Рашидович

Тестирование и отладка:

- Семеновская Елена Александровна
- Дмитриев Роман Николаевич

UX/UI-дизайн:

- Дмитриев Роман Николаевич

3. ЦЕЛЕВАЯ АУДИТОРИЯ

3.1. Основные группы пользователей

- Молодежь и студенты (14-25 лет) — для рефератов, презентаций и учебных работ.
- Контент-мейкеры / SMM (20-35 лет) — подбор цитат для постов в соцсетях.
- Преподаватели и лекторы (25-50 лет) — интеграция цитат в обучающие курсы и лекции.

3.2. Потребности аудитории

- Быстрый визуальный поиск цитат без регистрации.
- Автономный доступ к сохранённой базе.
- Удобная навигация и отзывчивый интерфейс.
- Образовательный интерактив (режим игры).

3.3. Сценарии использования

1. Ежедневный просмотр случайных цитат на главной панели.
2. Поиск по автору или ключевому слову через встроенный фильтр.
3. Использование игрового режима в учебных или развлекательных целях.
4. Массовое обновление коллекции с контролем прогресса и возможностью отмены.

4. Подбор средств для разработки

4.1. Язык программирования и платформа

- Язык: C# (версия 12)
- Платформа: .NET 8.0 (LTS)
- UI-фреймворк: Windows Presentation Foundation (WPF, net8.0-windows)

Обоснование: C# 12 предоставляет современную строгую типизацию, поддержку nullable-контекста и выразительный синтаксис. .NET 8 гарантирует высокую производительность JIT/AOT, кроссплатформенность базовых библиотек и долгосрочную поддержку. WPF выбран как промышленный стандарт для создания отзывчивых десктопных интерфейсов на Windows с богатой системой привязок данных и стилизации.

4.2. Среда разработки и инструменты сборки

- IDE: Microsoft Visual Studio 2022 / JetBrains Rider / VS Code с расширениями C# Dev Kit
- Сборка и запуск: dotnet CLI (dotnet build, dotnet run, dotnet publish)
- Управление зависимостями: NuGet (Microsoft.Data.Sqlite, System.Text.Json)
- Система контроля версий: Git + GitHub/GitLab

Обоснование: Современные IDE обеспечивают интеллектуальный анализ кода, отладку XAML, рефакторинг и интеграцию с Git. dotnet CLI унифицирует процесс сборки, тестирования и публикации приложения. NuGet позволяет быстро подключать проверенные библиотеки без ручного управления сборками.

4.3. Архитектурный паттерн и проектирование интерфейса

- Паттерн: MVVM (Model–View–ViewModel)
- Разметка: XAML, WindowChrome, словари ресурсов, Style/ControlTemplate
- Связывание данных: INotifyPropertyChanged, ObservableCollection<T>, RelayCommand (реализация ICommand)
- Оптимизация отрисовки: VirtualizingPanel, BooleanToVisibilityConverter

Обоснование: MVVM обеспечивает строгое разделение бизнес-логики, данных и визуального слоя, что упрощает модульное тестирование, параллельную работу команды и масштабирование. Виртуализация списков позволяет рендерить тысячи элементов без просадок FPS, а кастомный WindowChrome заменяет системный заголовок на тёмную панель в едином стиле приложения.

4.4. Работа с данными и сетевым взаимодействием

- Локальное хранилище: SQLite (Microsoft.Data.Sqlite)
- Сетевой клиент: HttpClient (статический экземпляр, Timeout = 60s, кастомный User-Agent)
- Сериализация: System.Text.Json (для ответов API и файла настроек)
- Асинхронность и управление состоянием: async/await, Task.Run, CancellationToken, IProgress<string>

Обоснование: SQLite — легковесная встраиваемая СУБД, не требующая отдельного сервера, поддерживающая транзакции и UNIQUE-дедупликацию на уровне БД. Статический HttpClient предотвращает исчерпание сетевых сокетов (SocketException). System.Text.Json оптимизирован по скорости и потреблению памяти. Асинхронные механизмы с токенами отмены и прогрессом гарантируют отзывчивость UI при длительных сетевых операциях и пакетной загрузке.

4.5. Вспомогательные инструменты и документирование

- Конфигурация: JSON-файл (settings.json), ручная сериализация с fallback-значениями
- Документация: Markdown (ARCHITECTURE.md), отчёт в формате DOCX
- Тестирование и отладка: Встроенный отладчик IDE, логирование статусов в UI, ручные сценарии проверки (сеть отключена, лимит API, отмена загрузки)

Обоснование: JSON-формат настроек удобен для хранения небольшого набора параметров, читаем человеком и легко модифицируется без перезапуска приложения. Markdown позволяет вести техническую документацию рядом с кодом, а DOCX соответствует стандартам оформления учебной проектной документации.

5. АНАЛИЗ АНАЛОГОВ

5.1. Существующие решения

1. *ZenquotesPy* (Python)

Легковесный Python-пакет для доступа к ZenQuotes API
GitHub

Преимущества: Простота использования

Недостатки: Только онлайн-режим, нет игровых функций

2. *Quote Generator Web App*

Веб-приложение для генерации цитат по жанрам
www.slideshare.net

Преимущества: Графический интерфейс

Недостатки: Требуется браузер, нет офлайн-режима

3. *AI Quote Generator (Mobile)*

Мобильное приложение с AI-генерацией цитат

Преимущества: Персонализация, визуальное оформление

Недостатки: Требуется установка, платные функции

4. *Quoootes.com*

Онлайн-генератор цитат с визуальным оформлением
quoootes.com

Преимущества: Создание графики для соцсетей

Недостатки: Только онлайн, нет API

5.2. Конкурентные преимущества ZenQuotesApp

Функция	Аналоги	ZenQuotesApp
Офлайн-режим	нет	есть (SQLite)
Кэширование	нет	есть (гибридное: API → БД → Backup)
Игровой режим	нет	есть («Угадай автора»)
Поиск по ключевым словам	частично	есть (мгновенный, регистронезависимый)
Современный UI	веб/мобильный	WPF MVVM, тема Spotify
Отмена и прогресс загрузки	нет	есть ('CancellationToken', 'IProgress')
Бесплатно	частично	полностью бесплатно
Резервный пул цитат	нет	есть (8 встроенных цитат)

5.3. Уникальные особенности

- MVVM-архитектура: чёткое разделение логики, данных и представления.
- Гибридная загрузка: каскадный фолбэк (Сеть → Локальная БД → Встроенный пул).
- Отказоустойчивость: сохранение уже скачанных данных при отмене или ошибке сети.
- Образовательный модуль: динамическая генерация вариантов ответов из реальной базы.
- Адаптивность к лимитам API: автоматические паузы при работе без ключа.

6. ПРОЦЕСС РАЗРАБОТКИ

6.1. Архитектура приложения

Проект построен по шаблону MVVM и разделён на независимые слои:

View (XAML) MainWindow.xaml + App.xaml — интерфейс, стили, привязки

| Binding / Commands

ViewModel MainViewModel, RelayCommand — состояние, команды, навигация

|

Services QuoteService, QuoteApiClient — бизнес-логика, сеть, настройки

SettingsStore, BackupQuotes

|

Data QuoteRepository — слой доступа к SQLite

|

Model Quote — структура данных

6.2. Основные компоненты

1. Модель данных (Quote.cs) – чистый POCO-класс (Text, Author), проходит через все слои.
2. Хранилище (QuoteRepository.cs) – SQLite (Microsoft.Data.Sqlite). UNIQUE по тексту + INSERT OR IGNORE гарантируют дедупликацию. Пакетные вставки в транзакции ускоряют запись в разы. БД хранится в %LOCALAPPDATA%\ZenQuotesApp\quotes.db.
3. API-клиент (QuoteApiClient.cs) – статический HttpClient с таймаутом 60 с и User-Agent. Десериализует JSON, фильтрует служебные ответы API, обрабатывает опциональный ключ.
4. Оркестратор (QuoteService.cs) – реализует стратегию загрузки: мультибатч → дедуп в Dictionary → сохранение в БД → фолбэк на БД/Backup. Поддерживает IProgress<string> и CancellationToken.
5. Настройки (AppSettings.cs, SettingsStore.cs) – сериализация в settings.json. При повреждении файла возвращаются значения по умолчанию.
6. ViewModel (MainViewModel.cs, ViewModelBase.cs, RelayCommand.cs) – управление ObservableCollection, навигация по разделам, асинхронные команды, отмена операций, валидация ввода.

7. View (MainWindow.xaml/cs) – кастомное окно (WindowChrome), боковая навигация, виртуализированные ListBox, статус-бар с прогрессом. Точка композиции зависимостей в Loaded.

6.3. Этапы разработки

Этап	Длительность	Выполненные работы
1. Проектирование	1 день	Выбор стека (.NET 8, WPF, MVVM, SQLite), проектирование слоёв, диаграмма потоков данных
2. Data & Services	1 день	`QuoteRepository` (SQLite, транзакции), `QuoteApiClient` (HTTP, JSON), `BackupQuotes`
3. Business Logic	1 день	`QuoteService` (фолбэк, паузы, прогресс), `SettingsStore`, обработка ошибок сети
4. ViewModel & Commands	1 день	`ViewModelBase`, `RelayCommand`, навигация, коллекции, асинхронная загрузка, отмена
5. View & UI/UX	2 дня	XAML-разметка, стилизация (Spotify-палитра), `WindowChrome`, виртуализация, статус-бар
6. Тестирование & Docs	1 день	Отладка фолбэков, проверка лимитов API, полировка UI, написание отчёта и `ARCHITECTURE.md`

6.4. Обработка ошибок

- Отсутствие интернета / таймаут → загрузка из SQLite.
- Пустая база → автоматическое использование BackupQuotes.
- Лимит API → пауза ~6 с между батчами без ключа.
- Повреждённые настройки → возврат к дефолтным значениям (BatchCount=3).

- Отмена загрузки → сохранение уже скачанных цитат в БД, корректный вывод статуса.

6.5. Оптимизации

- `async/await` + `Task.Run` для операций с БД → интерфейс не блокируется.
- `VirtualizingPanel` в `ListBox` → отрисовка только видимых элементов (тысячи цитат без лагов).
- Дедупликация на двух уровнях: `Dictionary` в памяти + `UNIQUE` индекс в `SQLite`.
- Статический `HttpClient` → предотвращение исчерпания сокетов.
- `CancellationToken` + `IProgress` → контролируемые долгие операции.
- Параметризованные SQL-запросы → защита от инъекций, корректная обработка кавычек.

7. MVP (MINIMUM VIABLE PRODUCT)

7.1. Определение MVP

Минимальная версия продукта, предоставляющая основную ценность: асинхронный доступ к цитатам, локальное хранение, графический интерфейс и базовая навигация.

7.2. Функции MVP

1. Отображение случайной цитаты на главном экране.
2. Локальное хранилище SQLite для офлайн-работы.
3. Асинхронная загрузка из API с прогрессом в статус-баре.
4. Навигация по разделам (Главная, Библиотека, Настройки).

7.3. Функции пост-MVP (реализовано)

1. Поиск по тексту и автору.
2. Игровой режим «Угадай автора» с динамическими вариантами.
3. Принудительное обновление с выбором количества батчей.
4. Встроенный резервный пул цитат.
5. Сохранение/загрузка настроек, отмена загрузки, кастомное окно.

7.4. Критерии успеха MVP

Критерий	Цель	Результат
Время запуска	< 3 сек	~1.5 сек
Количество цитат	≥ 50	~50+ за батч
Работа без интернета	Да	Полная (SQLite/Backup)
Интуитивность	< 1 мин на освоение	Навигация очевидна
Стабильность	Без падений	Graceful degradation + отмена

7.5. Критерии эффективности и метрики MVP

1. Отклик UI: < 16 мс (60 FPS) во время навигации и поиска.
2. Надёжность: ≥ 99% сессий без UnhandledException.
3. Покрывание сценариев: 100% ключевых путей (API→БД→Backup, отмена, поиск, игра).

8. ПРИМЕР ИСПОЛЬЗОВАНИЯ

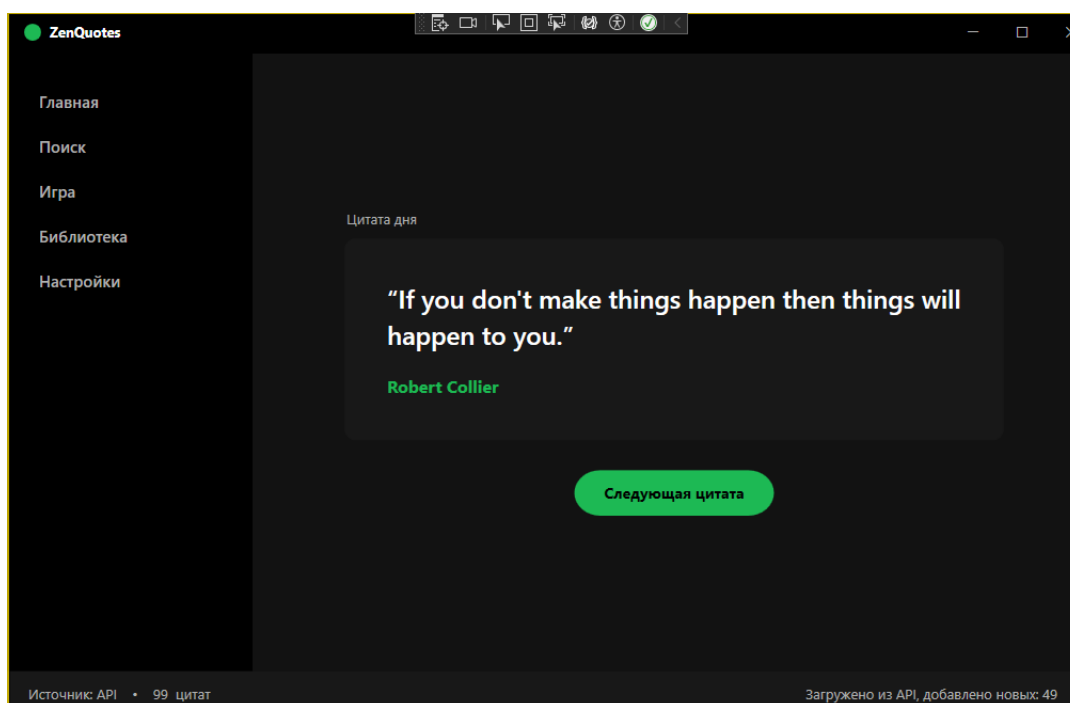
8.1. Установка и запуск

Требования: Windows 10/11, .NET 8.0 Runtime.

Запуск: `dotnet run` или запуск скомпилированного `ZenQuotesApp.Wpf.exe`.

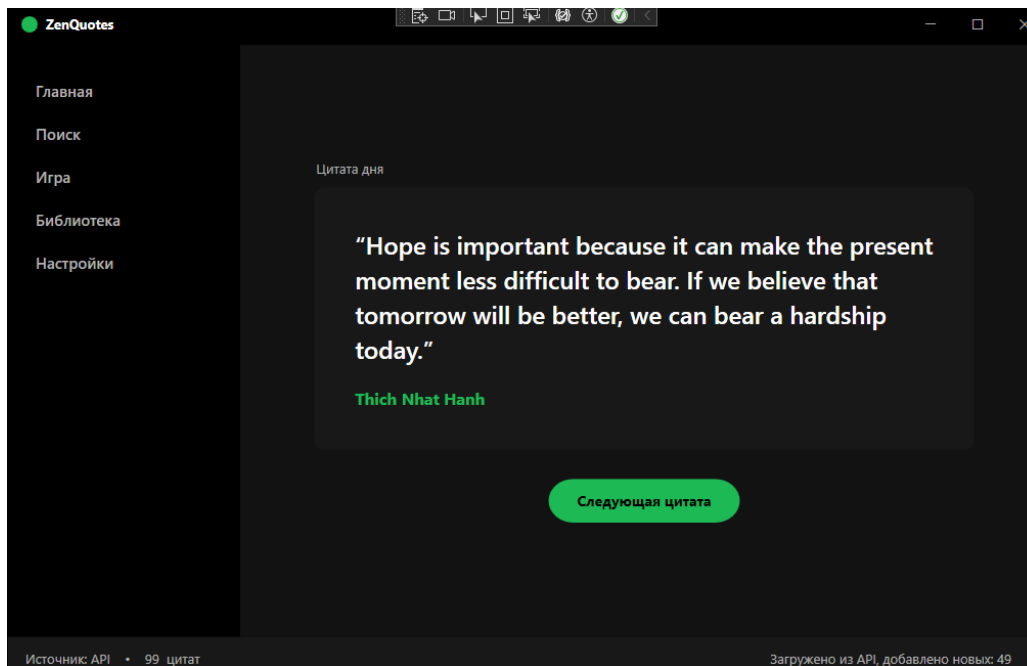
8.2. Сценарий 1: Первый запуск

Приложение автоматически создаёт `quotes.db` и `settings.json`. Запускается один запрос к API. В статус-баре отображается прогресс. При успехе цитаты сохраняются в БД и выводятся на главной панели. При ошибке – загружаются из резервного пула.



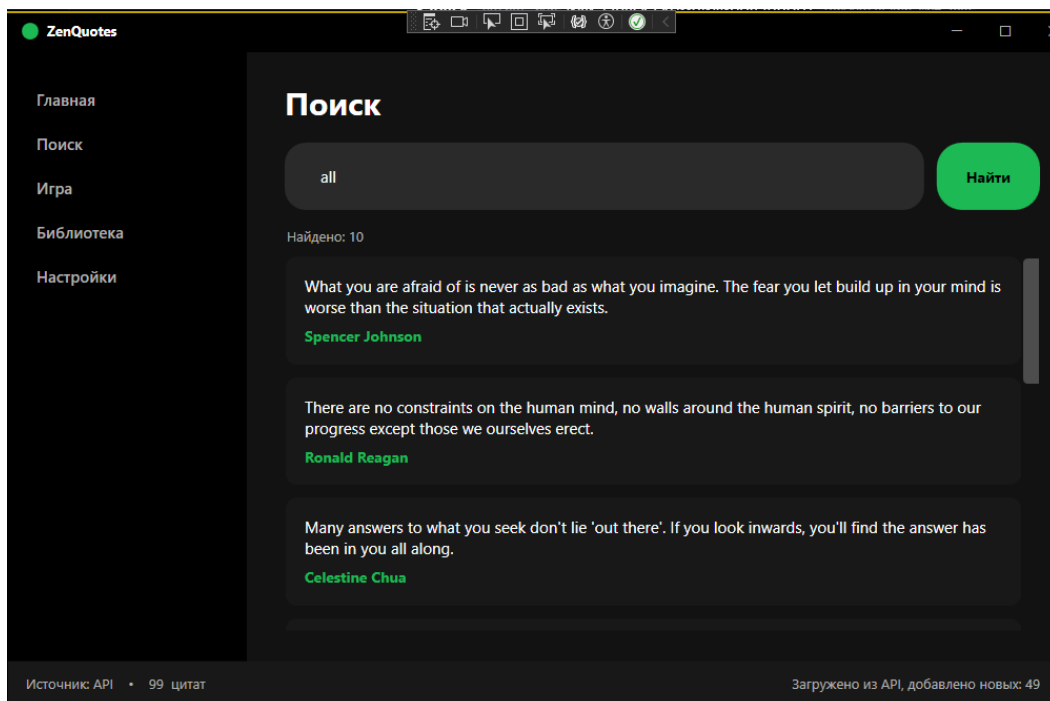
8.3. Сценарий 2: Просмотр случайной цитаты

На вкладке «Главная» отображается крупная карточка с текстом и автором. Кнопка «Следующая» мгновенно выбирает новую случайную цитату из загруженной коллекции без повторных запросов к сети.



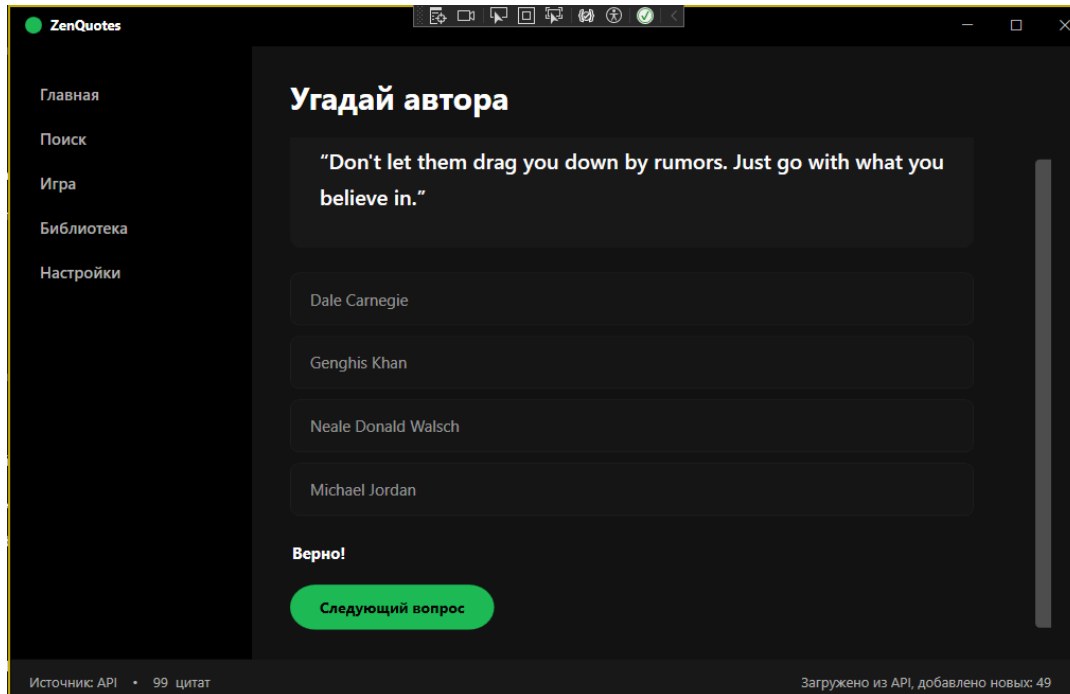
8.4. Сценарий 3: Поиск по ключевому слову

Вкладка «Поиск». Пользователь вводит слово в поле поиска → нажимает кнопку. Приложение фильтрует локальную коллекцию (регистронезависимо, по тексту и автору) и выводит результат. Статус показывает количество найденных элементов или сообщение об отсутствии совпадений.



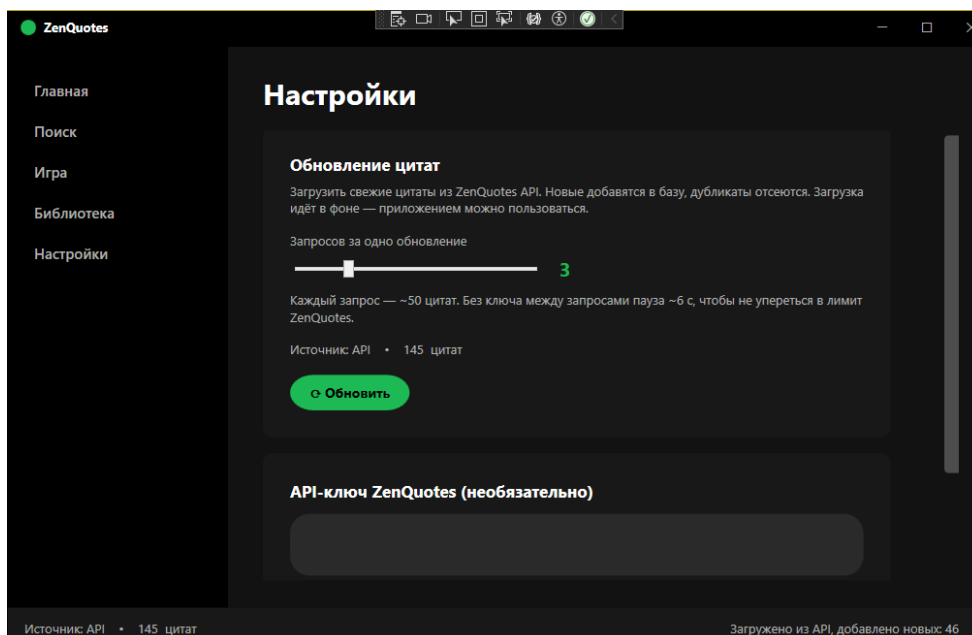
8.5. Сценарий 4: Игра "Угадай автора"

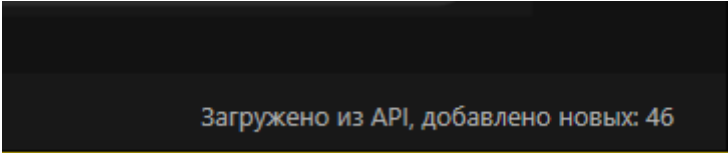
Вкладка «Игра». Приложение выбирает случайную цитату, скрывает автора и генерирует 4 варианта ответа (1 правильный + 3 случайных из других цитат). После нажатия на вариант появляется зелёный/красный фидбек. Кнопка «Новая игра» сбрасывает раунд.



8.6. Сценарий 5: Принудительное обновление

В настройках кнопка «Обновить», можно задать BatchCount (количество запросов подряд) и опциональный API-ключ. Во время загрузки статус-бар показывает текущий этап. Кнопка «Отмена» прерывает процесс, но уже скачанное сохраняется в БД.





Загружено из API, добавлено новых: 46

8.7. Сценарий 6: Работа без интернета

При отключении сети QuoteService получает исключение, автоматически переключается на чтение из quotes.db. Если база пуста (первый запуск без сети), подгружается BackupQuotes. UI остаётся полностью отзывчивым, загрузка не блокирует поток.

8.8. Структура файла хранилища

Файлы располагаются в %LOCALAPPDATA%\ZenQuotesApp\:

- quotes.db – SQLite база, таблица Quotes(Id, Text UNIQUE, Author).
- settings.json – JSON-файл настроек (BatchCount, ApiKey).

9. ЗАКЛЮЧЕНИЕ

9.1. Итоги разработки

Создано полнофункциональное десктопное WPF-приложение ZenQuotesApp на .NET 8. Реализована гибридная загрузка данных (API → SQLite → Backup), асинхронный интерфейс с прогрессом и отменой, игровой режим и гибкие настройки. Приложение устойчиво к сбоям сети и лимитам внешнего API.

9.2. Технические особенности проекта

- Архитектура MVVM с чётким разделением слоёв.
- SQLite с транзакциями и UNIQUE-дедупликацией.
- Асинхронные операции (async/await, Task.Run, CancellationToken, IProgress).
- Кастомный UI: WindowChrome, виртуализация списков, единая палитра стилей.
- Graceful degradation: приложение никогда не остаётся пустым экраном.

9.3. Практическая ценность

Проект демонстрирует применение современных практик C#-разработки: работа с REST-API, проектирование слоёв доступа к данным, построение отзывчивых WPF-интерфейсов, управление состоянием и асинхронностью. Код готов к использованию, расширению и интеграции в учебные или коммерческие проекты.

9.4. План дальнейшего развития проекта

- Добавить систему тегов/категорий и фильтрацию по темам.
- Реализовать «Избранное» с сохранением в отдельную таблицу SQLite.
- Экспорт цитат в изображение (PNG) для соцсетей.
- Адаптация под WinUI 3 / Avalonia для кроссплатформенности (Linux/macOS).
- Разработка Telegram-бота на основе текущей бизнес-логики.